

GridSentinel

Grid Load Optimisation & Renewable Energy Performance Advisor

Forecast → Detect → Diagnose → Optimise → Verify

Team CodeCatalyst · CHARUSAT

Heer Pathak (Team Lead) · Manasvee Viroja
Priyal Kalariya · Prince Patel

github.com/Pathakheer060628/bob-ai-hackathon-CodeCatalyst



Clean energy wasted, grid under stress

~8 TWh

of clean energy curtailed in the US in 2023

Solar and wind were switched off — not because the sun or wind stopped, but because the grid couldn't absorb the power at that moment.



Pressure 1 · Oversupply

Renewables produce more than the grid can take → clean power is curtailed (wasted).



Pressure 2 · Demand spikes

Unexpected load surges threaten stability — supply and demand must match within seconds.

Both pressures land on the same operator's desk — at the same time — with very limited real-time decision support.



Four jobs, one desk, fragmented tools



01

Forecast

Where is demand heading — and is it about to spike beyond plan?



02

Watch assets

Which solar farms and onshore / offshore wind assets are underperforming?



03

Diagnose why

Bad weather, equipment fault, or curtailment? Each needs a different response.



04

Decide & brief

Hour-by-hour dispatch, storage and demand response — then a 2-minute brief.

Today's reality

Fragmented tooling



Manual cross-referencing



Decisions under time
pressure



Wasted clean energy +
stability risk

GridSentinel: one pipeline, one verified brief

A single LangGraph pipeline over real grid data that forecasts, detects, diagnoses and optimises in sequence — and hands the operator one verified brief at the end.



Our golden rule

- Deterministic, unit-tested tools calculate every number.
- The LLM only narrates — it never calculates.
- A verifier re-checks every figure before an operator sees it.

Result: an AI advisor a grid operator can actually trust with real dispatch decisions.



Forecast demand spikes

Seasonal baseline + bounded trend



Detect renewable anomalies

CUSUM per asset class



Classify the root cause

Curtailment · weather · fault · surplus



Optimise load balancing

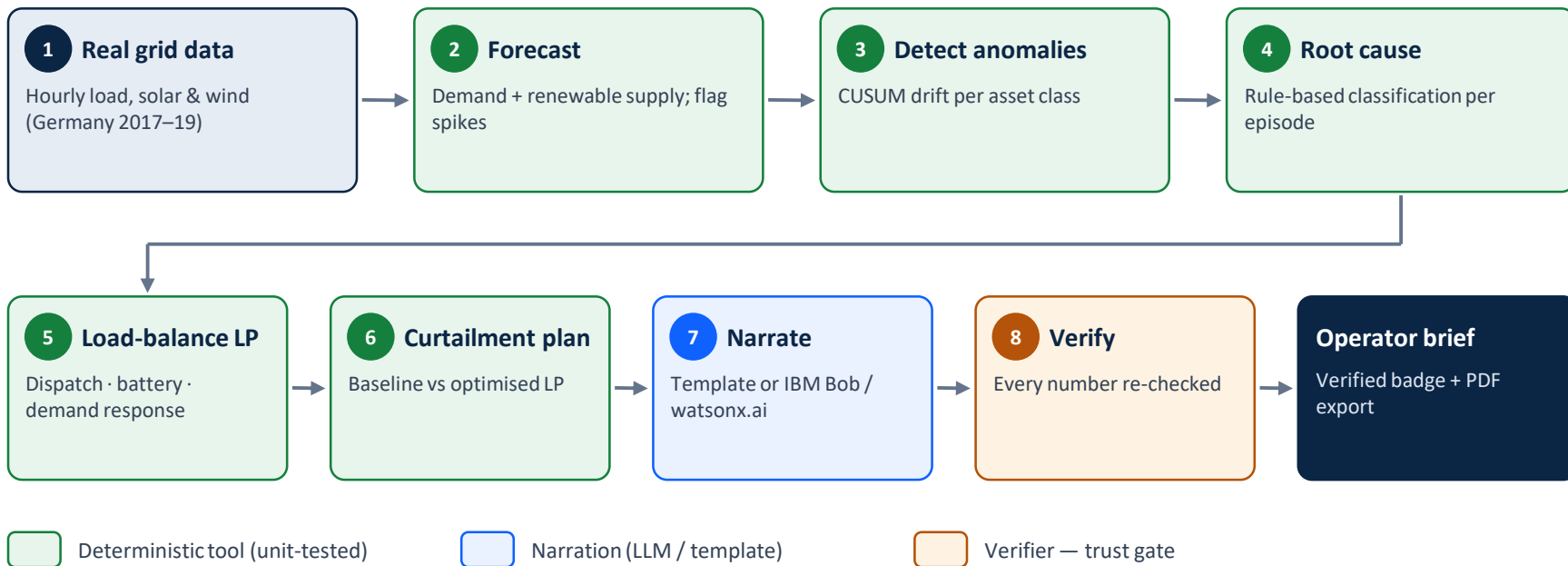
Linear program, hour by hour



Minimise curtailment

Baseline vs optimised → MWh saved

End-to-end pipeline: raw data → trusted brief

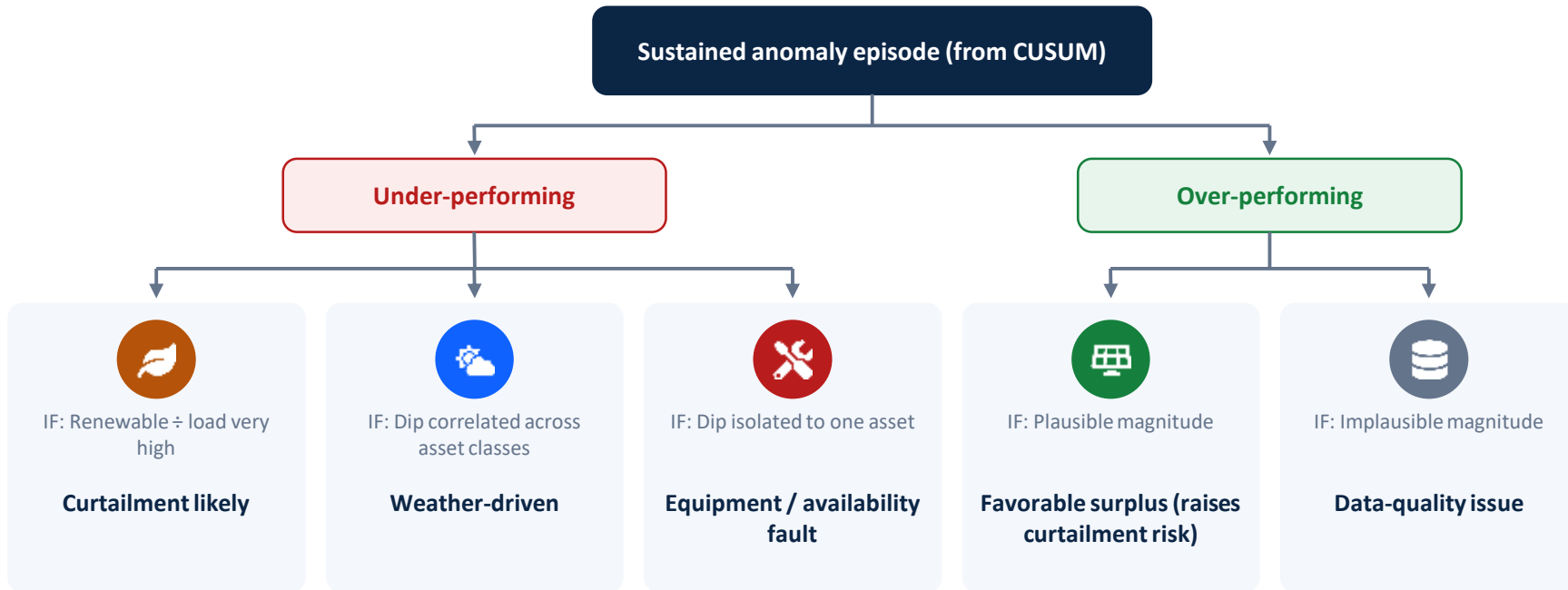


Orchestrated with LangGraph; every stage streams live progress to the dashboard via Server-Sent Events.

Every requirement, mapped to real code

Requirement	How GridSentinel does it	Where in the repo
Forecast demand spikes	Seasonal-naive baseline (median load for the same weekday/hour over a trailing window) + bounded trend adjustment. A spike = an hour unusual even after the normal weekly pattern.	<code>tools/forecasting.py</code>
Detect renewable anomalies	CUSUM change-point detection on (actual – expected) capacity factor, per asset class: solar, onshore wind, offshore wind. Sustained drift is flagged, single-hour noise is not.	<code>tools/anomaly_detection.py</code>
Identify root cause	Rule-based classifier using the renewable-to-load ratio and cross-asset correlation: curtailment-likely, weather-driven, equipment fault, or benign surplus.	<code>tools/root_cause.py</code>
Recommend load balancing	Linear program (scipy linprog) choosing dispatch, battery charge/discharge and demand response per hour to meet forecast demand at minimum cost.	<code>tools/load_balancing.py</code>
Minimise curtailment	Runs the LP twice — no-flexibility baseline vs fully configured plan — and reports the MWh difference as avoided curtailment.	<code>tools/curtailment.py</code>
Integrated, trusted brief	LangGraph orchestrator wires all stages; narration provider writes the brief; verifier checks every number.	<code>agents/orchestrator.py</code> <code>agents/verifier.py</code>

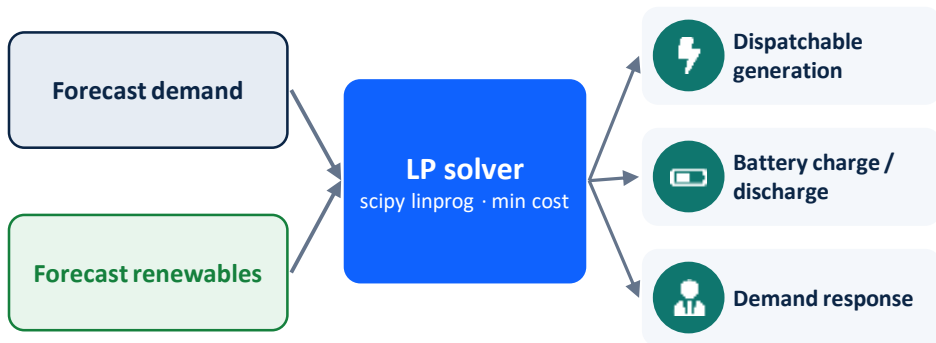
Why is an asset off? Transparent root cause



No model call — every label is explainable and unit-tested (tests T22–T26). Each cause triggers a different operator response.

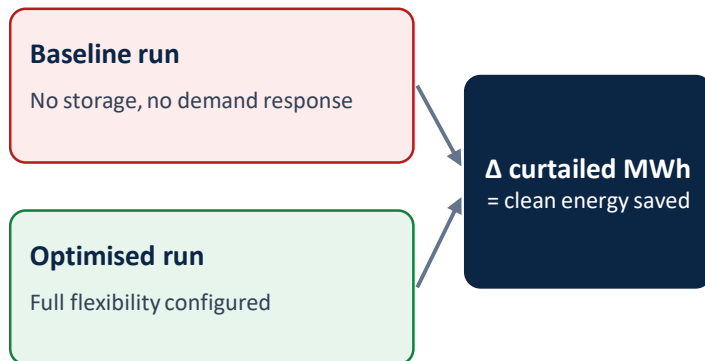
Balance every hour — and prove the saving

A · Load-balancing linear program



Solved jointly for every hour of the forecast horizon.

B · Curtailment minimisation

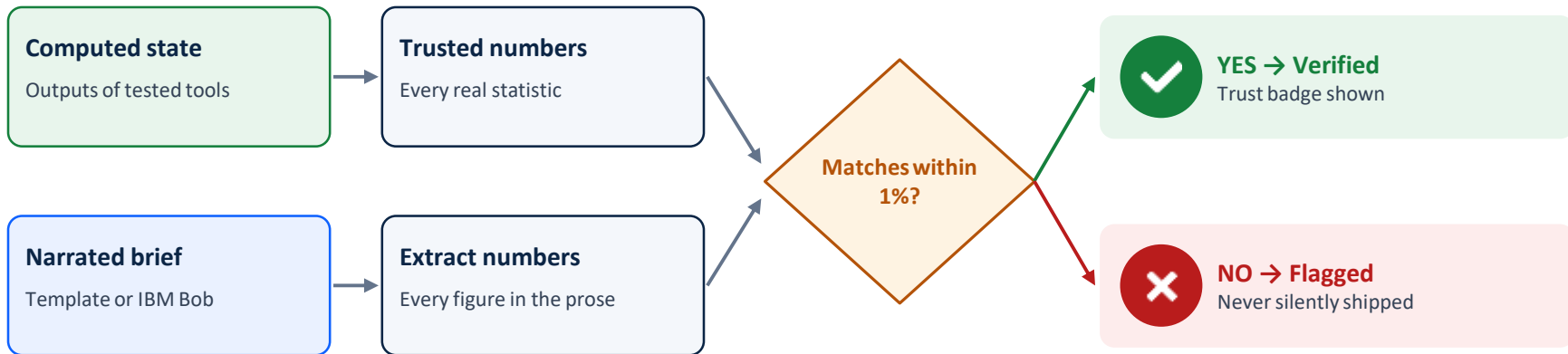


Isolates the real operational value of flexibility — not just a raw curtailment number.



Proven by tests: supply-demand balance holds every hour when feasible · storage absorbs oversupply instead of curtailing · optimised curtailment never exceeds baseline · unmet demand is reported honestly when capacity is insufficient.

The Verifier: no unchecked number ships



LLM narrates, never calculates

Every forecast, anomaly, LP and curtailment figure comes from a typed, unit-tested tool.



Hallucination test built in

Test T40 injects a fabricated number into a real brief — the verifier catches it.



Same scrutiny for any provider

An IBM Bob-written brief is checked exactly like the offline template.

System architecture & tech stack

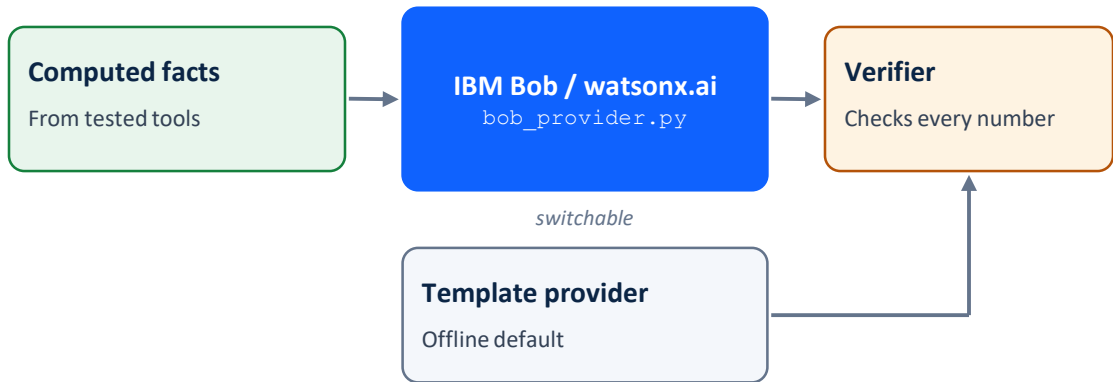
Presentation	React 18 + Vite dashboard — live progress feed, verifier trust badge, one-click PDF
API	FastAPI — REST endpoints + Server-Sent Events (live stage-by-stage progress)
Orchestration	LangGraph pipeline (orchestrator.py) + verifier agent (verifier.py)
Analytics tools	pandas · numpy · scipy linprog — forecasting, CUSUM, root cause, LP, curtailment
Narration & reports	Offline template provider (default) · IBM Bob / watsonx.ai provider · Jinja2 + ReportLab PDF
Data	Real hourly German load, solar & wind 2017–2019 (Open Power System Data / ENTSO-E)

Built to be reproduced

- Runs fully offline — no API key needed
- Real dataset committed to the repo
- Python 3.11 · pytest · one-click PDF

```
# backend
cd src
pip install -e ".[dev]"
uvicorn backend.api.main:app \
    --reload --port 8000
# frontend
cd src/frontend
npm install && npm run dev
```

Where IBM Bob / watsonx.ai fits in GridSentinel



Why this design works

- Pluggable: swap template ↔ IBM Bob without touching any math
- Bob receives structured, pre-computed facts → readable operator prose
- Bob's output passes the same verifier — no hallucinated figures



Transparent status (as stated in our README)

- The IBM Bob / watsonx.ai narration provider is real, wired code (src/backend/llm/bob_provider.py).
- It was not exercised in this build — no watsonx API key was available; the demo runs on the offline template provider, which produces the same facts.
- Plugging in credentials activates Bob-written briefs — with the verifier guarding every number.

Validated end to end: 50 / 50 tests passing

50/50

tests passed

94%

backend statement coverage

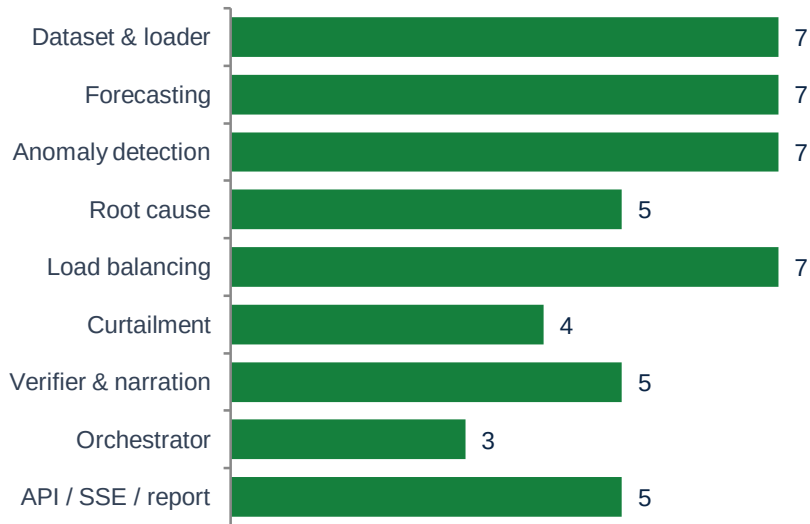
5.64 s

full suite run time

0

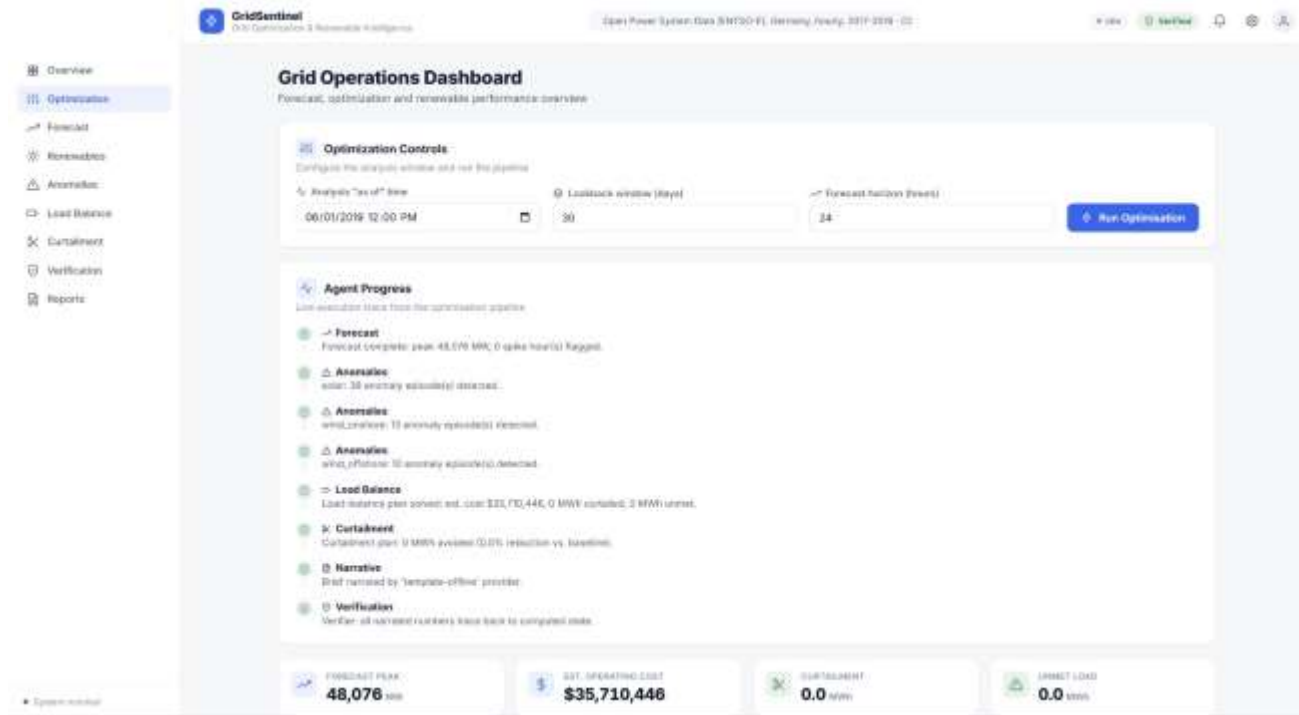
failures · skips · errors

Passing tests by category (9 categories)



Source: TEST_REPORT.md in our repo — pytest with coverage, run from src/backend. Verified: CUSUM ignores single-hour blips, verifier catches a fabricated number, full LangGraph pipeline runs on real data.

Live demo: the running GridSentinel dashboard



Screenshots captured from the actual running app in a live browser (demo/screenshots/in our repo).

Real data, honest limits



Real data, not synthetic

- Genuine hourly German grid load, solar and wind generation, 2017–2019
- Source: Open Power System Data, built from ENTSO-E TSO reports
- Contains real demand spikes, multi-day wind lulls and surges, and real forecast-vs-actual gaps

Known limitations — stated openly



Curtailment is near-zero in most windows of this dataset

At national scale, renewables rarely exceeded demand. The LP is verified on synthetic oversupply tests and reports savings automatically on high-renewable windows.



IBM Bob provider wired, not exercised

No watsonx API key in the build environment; demo uses the offline template provider.



Run storage is in-memory

Correct for the demo; a real database is needed for multi-user use.



Load-balancing config not yet a UI control

Defaults come from each window's peak load; the API already accepts overrides.

From hackathon build to control-room copilot

Impact for grid operators



One verified brief instead of spreadsheets to reverse-engineer



Curtailment saved, quantified in MWh — the value of flexibility made visible



Explainable root causes → the right response for weather vs fault vs curtailment



Every number traceable to a tested computation — safe for real dispatch decisions

Roadmap



NOW

Verified offline pipeline on real grid data



NEXT

Live IBM Bob / watsonx.ai narration with credentials



THEN

Persistent database + UI controls for storage, dispatch & demand response



SCALE

Higher-renewable grids and other regions' datasets — where curtailment savings grow

Thank you

GridSentinel — forecasts the grid, finds what's wrong, explains why, plans the fix, and proves every number.

Team CodeCatalyst · CHARUSAT

Heer Pathak (Lead) · Manasvee Viroja · Priyal Kalariya · Prince Patel

github.com/Pathakheer060628/bob-ai-hackathon-CodeCatalyst

Questions welcome.

